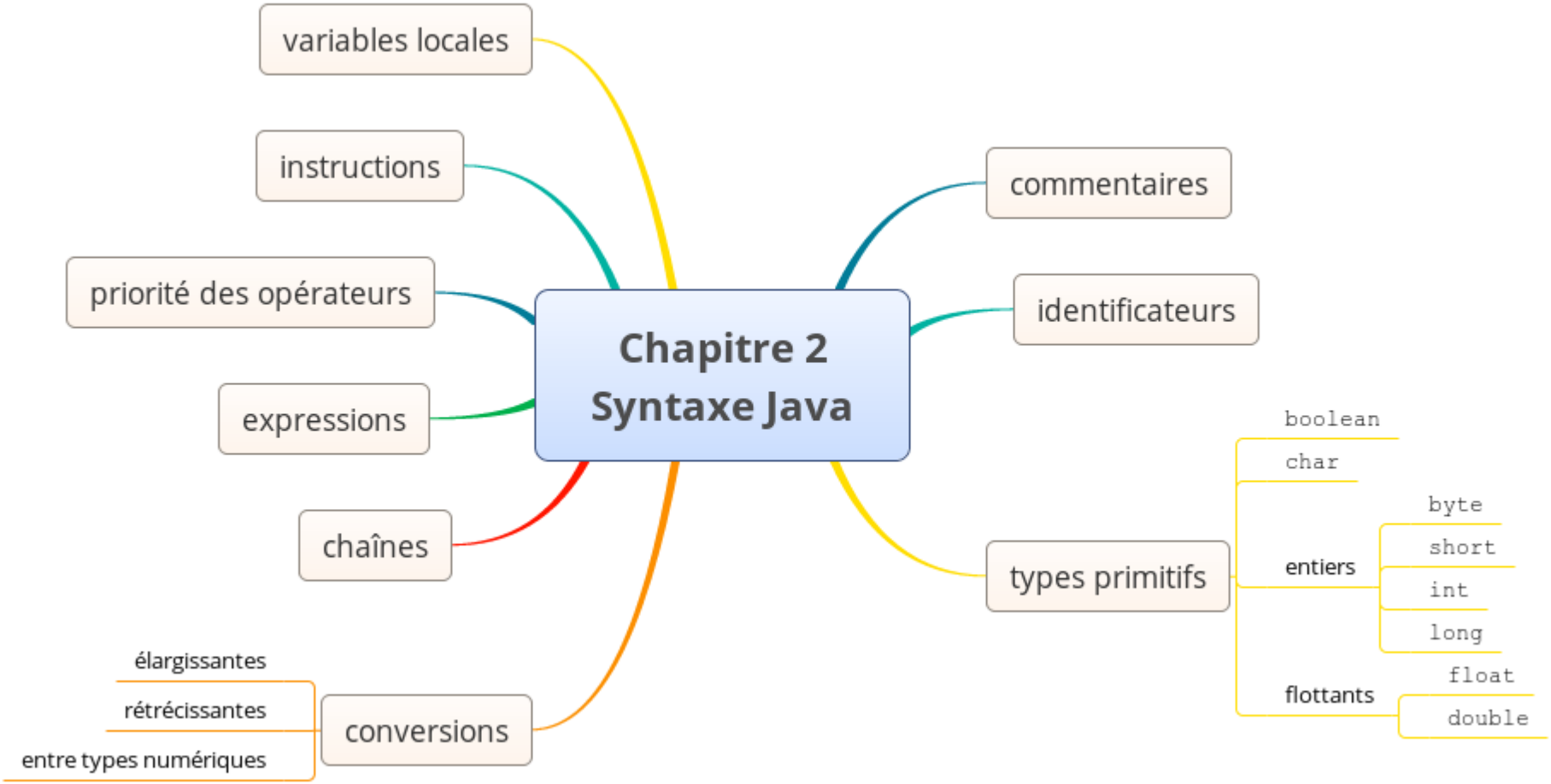
Séance 2 / 12


Chapitre 2
Syntaxe Java
variables locales
instructions
priorité des opérateurs
expressions
chaînes
conversions
élargissantes
rétrécissantes
entre types numériques
commentaires
identificateurs
types primitifs
boolean
char
entiers
byte
short
int
long
flottants
float
double

Commentaires

- 3 types de commentaires

```
/*
 * Commentaire simple qui s'étend
 *  sur plusieurs lignes.
 */

// Commentaire de fin de ligne

/**
 * Commentaire de documentation
 *  réservé au générateur automatique
 *  de documentation (outil javadoc).
 */
```

Identificateurs

- Construction :
  - lettres (+ lettres accentuées)
  - chiffres
  - `$` et `_`
- `a` ≠ `A`
- Longueur illimitée
- `$` utilisé par le compilateur → prudence !

Types primitifs

- 8 types primitifs pour travailler

Type	Valeurs	Taille (en bits)
boolean	`false, true`	?
char	`'\u0000'` → `'\uFFFF'`	16
byte	`-128` → `127`	8
short	`-32768` → `32767`	16
int	`-2147483648` → `2147483647`	32
long	`-9223372036854775808` → `9223372036854775807`	64
float	`1.4E-45` → `3.4028235E38`	32
double	`4.9E-324` → `1.7976931348623157E308`	64

- “Type primitif” par opposition à “Type objet”

Type `boolean`

- Seulement 2 valeurs : `false` et `true`
  - ! différence avec le langage C

beug sournois ← En C

```
int x, y;
...
if (x = y) {
    // y != 0
    // x == y
    ...
} else {
    // y == 0
    // x == y
    ...
}
```

En Java → erreur à la compilation

```
int x, y;
...
if (x = y) {
    ...
} else {
    ...
}
```

Type `char`

- Caractères Unicode → 16 bits non signés
  - Caractères normaux
    - `'A'`
    - `'\101'` (A en octal)
    - `'\u0041'` (A en Unicode)
  - Caractères spéciaux :
    - `'\b'` (espace arrière)
    - `'\t'` (tabulation)
    - `'\n'` (nouvelle ligne)
    - `'\f'` (page suivante)
    - `'\r'` (retour chariot)
    - `'\"'` (guillemet)
    - `'\''` (apostrophe)
    - `'\\'` (barre oblique inverse)

Types entiers

- Littéraux en base 10 (`65`) ou base 8 (`0101`) ou base 16 (`0x41`)
- Entiers simple précision
  - `65` littéral de type **`int`**
- Entiers double précision
  - `65L` littéral de type **`long`**
- Calculs réalisés sur les **`int`** ou les **`long`**

```
byte b = 1;
b = b + b;
```

Conversion automatique sur le littéral entier `1`

Erreur car `b + b` est de type **`int`**

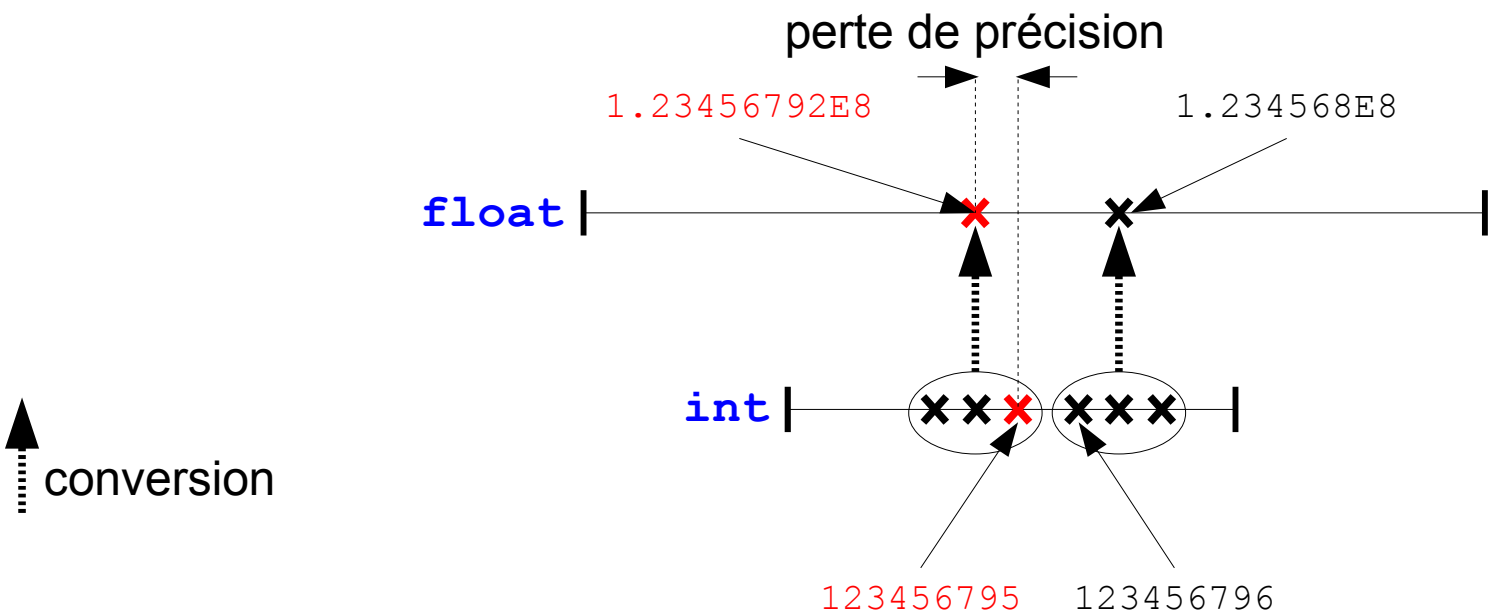
Il faut coder `b = (byte) (b + b);`

Types flottants

- **`float`** (simple précision)
  - environ 7 chiffres significatifs
  - $1.4\ 10^{-45} \leq |x| \leq 3.4\ 10^{+38}$
  - `3.14f` littéral de type **`float`**
- **`double`** (double précision)
  - environ 17 chiffres significatifs
  - $4.9\ 10^{-324} \leq |x| \leq 1.8\ 10^{+308}$
  - `3.14` ou `3.14d` littéral de type **`double`**

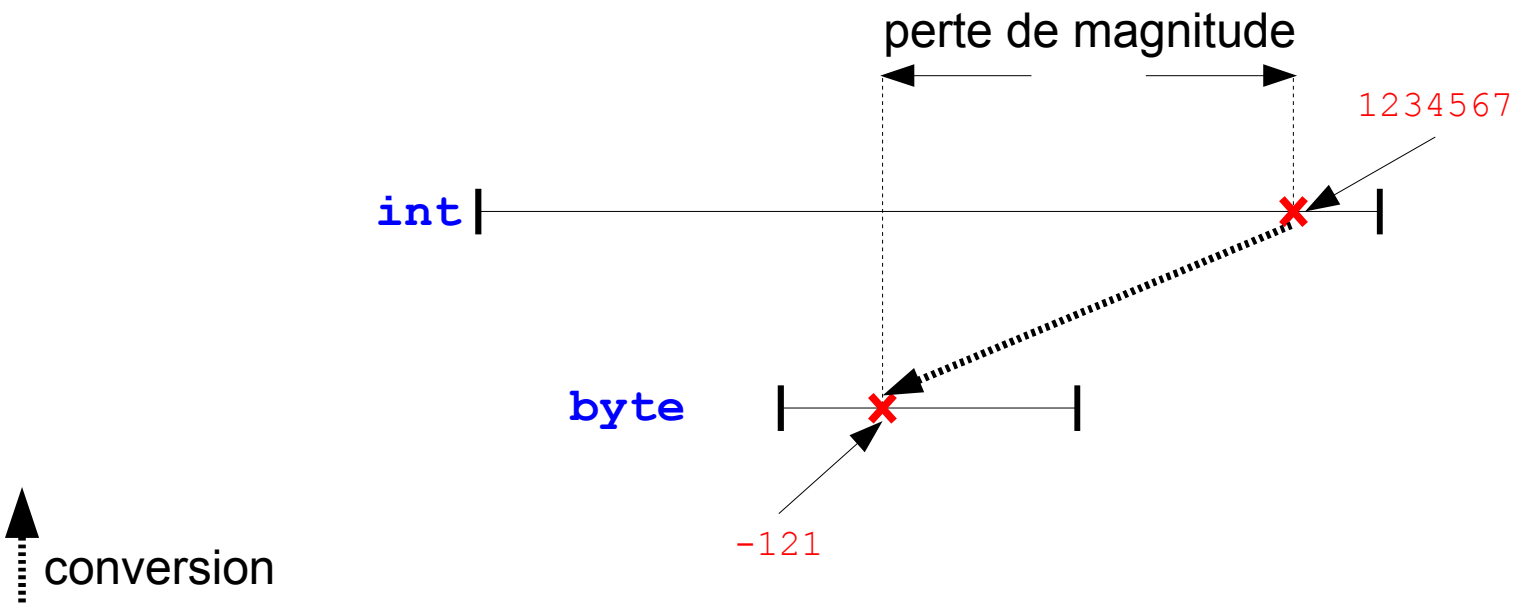
Conversion : perte de précision

```
float x = 123456795
```



Conversion : perte de magnitude

```
byte b = (byte) 1234567
```



Conversions élargissantes

- **`byte`** → **`short`**, **`int`**, **`long`**, **`float`**, **`double`**
- **`short`** → **`int`**, **`long`**, **`float`**, **`double`**
- **`char`** → **`int`**, **`long`**, **`float`**, **`double`**
- **`int`** → **`long`**, **`float`**, **`double`**
- **`long`** → **`float`**, **`double`**
- **`float`** → **`double`**

- entier → entier : aucune perte de précision
- entier → flottant : perte de précision possible
- ne lèvent jamais d'exception

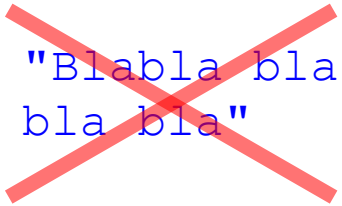
Conversions rétrécissantes

- **`short`** → **`byte`**, **`char`**
- **`char`** → **`byte`**, **`short`**
- **`int`** → **`byte`**, **`short`**, **`char`**
- **`long`** → **`byte`**, **`short`**, **`char`**, **`int`**
- **`float`** → **`byte`**, **`short`**, **`char`**, **`int`**, **`long`**
- **`double`** → **`byte`**, **`short`**, **`char`**, **`int`**, **`long`**, **`float`**

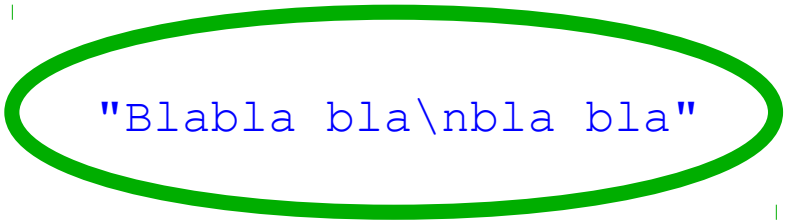
- perte de précision possible
- perte de magnitude possible
- ne lèvent jamais d'exception

Chaînes de caractères

- C'est un type objet
- Littéraux entre guillemets
  - `"BlablA Blabl\101 Blabl\u0041"`
- Pas de retour à la ligne



NON



OUI

Expressions d'affectation

identificateur de variable → `v = e` ← expression compatible

- Affectation simple = Expression à effet de bord dont la valeur est celle de `e`
  - effet de bord = modification de la valeur de `v`
  - expression ≠ instruction
- Affectations multiples : `i = j = k = 2`
  - opérateur associatif à droite
    `k = 2; j = 2; i = 2;`
- Autres expressions d'affectation
  - `*=`, `+=`, `-=`, `/=`, ...
  - (`v += e` ⇔ `v = v + e`)

Exemples d'expressions

`! true`	➢	`false`
`10 == 10/3 * 3 + 10%3`	➢	`true`
`10.2 / 2 == 5.1`	➢	`true`
`// i == α` `(i > 0) ? 1 : -1`	➢	`1 si α > 0, -1 sinon`
`2 + " donuts"`	➢	`"2 donuts"`
`i = 3` `// i == 3`	➢	`3`

`// i == α`
`i *= 2`
`// i == 2α`
	➢	2α (équivaut à `i = i*2`)

`(i *= i) == (i = i*i)`	➢	`false`

`// i == α`
`++i`
`// i == α+1`
	➢	α+1

`// i == α`
`i++`
`// i == α+1`
	➢	α

Priorité des opérateurs

Principaux opérateurs

Priorité	Opérateur	Arité	Type	Description
1	`++`, `--`, `+`, `-` `!`, `~`	1	arithmétique booléen	incrémentation décrémentation + et - unaires compléments logique et bit à bit
2	`*`, `/`, `%`	2	arithmétiques	mult, div, mod
3	`+`, `-` `+`	2	arithmétiques `String`s	add, sub concaténation
5	`<`, `<=`, `>`, `>=`	2	arithmétiques	comparaison
6	`!=`, `==`	2	TT, TT	différence, égalité
7	`&`	2	booléens	ET logique
8	`^`	2	booléens	XOR
9	`|`	2	booléens	OU logique
10	`&&`	2	booléens	ET conditionnel
11	`||`	2	booléens	OU conditionnel
12	`? :`	3	booléen, TT, TT	opérateur conditionnel
13	`=`, `*=`, `/=`, `%=`, `+=`, `-=`	2	TT, TT	affectations

TT : Tout Type

Instructions (1/10)

- <u>Bloc d'instructions</u>

  Exécute en séquence `I1`, puis `I2`, ..., puis `In`

```
{
    I1
    I2
    ...
    In
}
```

- <u>Instruction vide</u>

```
;
```

  Ne fait rien

Instructions (2/10)

- <u>Instructions simples</u>
  - <u>À base d'expression</u>

    ```
    <expression>;
    ```

    Évalue l'expression (effet de bord éventuel) puis ignore la valeur obtenue

affectation	`x = 2;`
opérateur `++` ou `--`	`i++;`
appel de méthode	`<obj>.<méth>(<args>);`
instanciation	`new NomClasse(<args>);`

  - <u>À base de déclaration</u>

    ```
    <déclaration>;
    ```

    Effectue la déclaration (initialisation éventuelle)

variable	`int x;`
variable initialisée	`int y = 2;`
constante	`final double pi = 3.14159;`

Instructions (3/10)

- <u>Instruction if-else</u>

```
if (<expr bool>) I1 [else I2]
```

Évalue `<expr bool>` puis exécute `I1` si le résultat est **`true`** ou `I2` si le résultat est **`false`**.

```
if (x >= 0) {
    y = x;
} else {
    y = -x;
}
```

Instructions (4/10)

```
switch (<expression>) {
    {case <expr>: {I}*}*
    [default : {I}*]
}
```

- <u>Instruction switch</u>

  Évalue `<expression>` (de type **`char`**, **`byte`**, **`short`** ou **`int`**)
  puis toutes les `<expr>` (deux à deux distinctes)

  Ensuite compare `<expression>` avec chaque `<expr>`
  (de la première à la dernière) :

  - S'il existe `<expri>` telle que `<expression> == <expri>`
    alors exécute toutes les instructions entre `:` et la dernière `}` du switch
  - Sinon si **`default`** présent
    alors exécute toutes les instructions entre `:` et la dernière `}` du switch
  - Sinon aucune instruction exécutée

```
switch (2*i) {
    case 4:
        System.out.print("beaucoup ");
    case 1+1:
        System.out.println("trop");
}
```

tester pour
`i == 0`
`i == 1`
`i == 2`

Instructions (5/10)

- <u>Instruction while</u>

```
while (<expr bool>) I
```

  - Évalue `<expr bool>`
    - Si **`true`** : exécute `I` et recommence
    - Si **`false`** : s'arrête

```
int i = 0;
while (i < n) {
    System.out.print(((i == 0) ? "" : " ") + t[i]);
    i += 1;
}
```

Rq : on suppose `t` un tableau d'au moins `n` éléments

Instructions (6/10)

- <u>Instruction do-while</u>

```
do I while (<expr bool>);
```

  - Exécute `I` et évalue `<expr bool>`
    - Si **`true`** : recommence
    - Si **`false`** : s'arrête

```
int i = 0;
do {
    System.out.print(((i == 0) ? "" : " ") + t[i]);
    i += 1;
} while (i < n);
```

Rq : on suppose `t` un tableau d'au moins `n` éléments

Instructions (7/10)

- <u>Instruction for</u> `for ([<expr init>]; [<expr bool>]; [<expr màj>]) I`
  - Évalue `<expr init>`
  - (a) Évalue `<expr bool>`
    - Si **`true`** : exécute `I` puis `<expr màj>` et recommence en (a)
    - Si **`false`** : s'arrête

```
for (int i = 0; i < n; i += 1) {
    System.out.print(((i == 0) ? "" : " ") + t[i]);
}
```

Rq : on suppose `t` un tableau d'au moins `n` éléments

Instructions (8/10)

- <u>Instruction break</u> `break;`

  Termine immédiatement l'instruction (**`switch`**, **`while`**, **`do`** ou **`for`**) à l'intérieur de laquelle elle se trouve

```
switch (i) {
    case 0:  System.out.println("Y en a plus !"); break;
    case 1:  System.out.println("Y en a plus qu'un !"); break;
    default: System.out.println("Y en a encore plein..."); break;
}
```

Instructions (9/10)

- <u>Instruction continue</u> `continue;`

  Termine immédiatement l'itération courante et entame la suivante (dans **`while`**, **`do`** ou **`for`**)

```
for (int i = 0; ; i++) {
    if (i == 5) {
        System.out.print("hop et "); continue;
    } else if (i == 10) {
        System.out.print("c'est tout !"); break;
    }
    System.out.print(i + " et ");
}
System.out.println();
```

Affiche : `0 et 1 et 2 et 3 et 4 et hop et 6 et 7 et 8 et 9 et c'est tout`

Instructions (10/10)

- <u>Instruction return</u>

```
return [<expr>];
```

Termine immédiatement l'exécution d'une procédure (sans `<expr>`) ou d'une fonction (avec `<expr>`)

```
void afficherDiv(int x, int y) {
    if (y == 0) {
        return;
    }
    System.out.println(x / y);
}
```

Affiche la valeur de `x/y` quand `y != 0`

```
String miroir(String s) {
    if (s.equals("")) {
        return "";
    }
    return
        miroir(s.substring(1))
        + s.charAt(0);
}
```

Sur l'argument `"bonjour"` retourne `"ruojnob"`

Déclaration de variable locale

```
[final] <type>{[]}* <id>[ = <exp>]{, <id>[ = <exp>]}*
```

- <u>Variable locale</u> définie à l'intérieur d'un bloc par une déclaration, avec ou sans initialisation.

```
int x = 5
```

```
int a, b = 3, c = b*2
```

?	3	6

```
int[] d, e = new int[3]
```

?	0	0	0

```
int[][] t = new int[][] {{1, 2, 3}, {4, 5, 6}}
```

1	2	3
4	5	6

```
final float pi = 3.14f
```

- Portée d'une variable locale limitée au bloc de définition
- Exemple :
  -
```
int n = 0;
while (n < 10) {
    int x = 5;
    ...
    n += 1;
}
System.out.println(x);
```

variable locale au corps du **while**

erreur (compilation)
`x` n'est pas accessible ici

Règles de déclaration des variables locales

- Toute variable locale doit être *déclarée et initialisée avant d'être utilisée*
- Une déclaration peut survenir *n'importe où dans un bloc*
- Une variable est visible *à l'intérieur du bloc* dans lequel elle est déclarée, *à partir de sa déclaration*
- Une déclaration *ne peut pas en masquer une autre*
- Une constante (variable déclarée avec `final`) ne peut *prendre valeur qu'une seule fois*

Annexe
Références bibliographiques

Sur la toile :

Page de ressources de ce cours
http://dpt-info-sciences.univ-rouen.fr/~andarphi/lbb/

The Java Tutorial : Learning the Java Language. Oracle.
http://docs.oracle.com/javase/tutorial/java/TOC.html

Java SE Development Kit Documentation 1.4.2
http://www.oracle.com/technetwork/java/javaseproducts/downloads/
java-archive-downloads-javase14-419411.html#7502-j2sdk-1.4.2-doc-oth-JPR

The Java language Specification (Third Edition). James GOSLING, Bill JOY, Guy STEELE
http://docs.oracle.com/javase/specs/jls/se6/html/j3TOC.html

Sur le papier :

Conception objet en Java avec Bluej (4° éd.)
David J. BARNES, Michael KÖLLING
Pearson Education, 2013 (ISBN : 978-2326001176)

Le langage C ANSI, vers une pensée objet en Java (3° éd.)
Philippe DRIX
Dunod, 2000 (ISBN : 978-2100051939)

Conception et programmation orientées objet
Bertrand MEYER
Eyrolles, 2008 (ISBN : 978-2212122701)

THE baïbeul

Coder proprement
Robert MARTIN
Pearson, 2009 (ISBN : 978-2744023279)

THE bèdsaïde bouc